

Architecture Selection

We chose **Layered Architecture** for the **Hostel Maintenance & Issue Ticketing System**.

Architectural Choice

The system is organised into three layers. The Presentation layer holds the Ticket UI. The Business layer holds the Ticket Manager, Authentication Service, Dispatch & SLA Service and Notification Service. The Data / Infrastructure layer holds the Database Component and the Attachment Store. Components communicate only through provided / required interfaces (ITicketAPI, IAuth, IDispatch, INotify, IStorage, IData), so any component can be replaced without changing its callers.

Two Reasons for the Choice

1. **Scale and simplicity of the scenario.** A hostel serves one campus with a few hundred residents and one warden team. One deployable application with three layers is enough, cheap to host, and easy for a small student team to build, test and maintain; microservices would add network, deployment and monitoring overhead with no benefit.
2. **Ticket data needs strong consistency.** Every ticket must get a unique ID, follow ordered status changes (Open > Assigned > Resolved > Closed) and carry accurate SLA timestamps. A single shared database behind the Ticket Manager guarantees this, whereas splitting data across services would create distributed-consistency problems for FR-001 to FR-005.

Security Advantage

Layers give one controlled path to the data. The Ticket UI never talks to the database or file store directly; every request goes through the Authentication Service and the Ticket Manager, where role-based access (Resident vs Warden) is enforced, so a resident can only see their own tickets (FR-003, NFR-002). All traffic uses HTTPS/TLS, and only the data layer uses parameterised SQL, which limits SQL-injection risk.

Performance Benefit

Because the SLA timer logic is isolated in the Dispatch & SLA Service and alerts are sent by a separate Notification Service, escalation checks (NFR-001) run in the background without slowing residents who are logging tickets. Photos are kept in the Attachment Store instead of the database, keeping ticket queries small and fast, and the stateless business layer can be run as several instances behind a load balancer if demand grows.

Comparison of Styles Considered

Style	Pros for this system	Cons for this system	Verdict
Layered	Clear UI / logic / data split; simple to build and test	Slight call overhead between layers	Selected
Microservices	Independent scaling and deployment	Operational complexity, network latency, distributed data for a single-hostel workload	Rejected: over-engineered
Client-Server	Central control, easy consistency	Single point of failure; little internal structure	Layered is a refinement of it